الدليل الفني الشامل لإدارة Amazon EKS

EKS Auto Mode | Authentication Modes | Endpoint Access | Node IP Sizing

1. وضع الإدارة التلقائية (EKS Auto Mode)

خيار **EKS Auto Mode** هو ميزة حديثة تقدمها AWS لإدارة البنية التحتية لـ Kubernetes بشكل مؤتمت بالكامل. يوسع هذا الوضع مسؤوليات AWS من إدارة الـ Control Plane فقط ليشمل إدارة المكونات التشغيلية لـ Worker Nodes، والشبكات (Networking)، والتخزين (Storage).

ماذا يفعل EKS Auto Mode بالتفصيل؟

- إدارة الـ **Compute:** يتولى إنشاء وتوسعة (Auto-scaling) وحذف الـ EC2 Instances تلقائياً بناءً على متطلبات الـ Pods باستخدام تقنيات شبيهة بـ Karpenter، دون الحاجة لإنشاء Node Groups يدويين.
- **تحديثات الأمان:** يقوم بترقية نظام التشغيل (OS) والـ Kubelet وتطبيق ثغرات الأمان (Patching) على العقد بأسلوب Rolling Updates بدون توقف الخدمة.
- إدارة **Storage & CNI:** تثبيت وتحديث محركات التخزين (EBS CSI Driver) وتكامل شبكات VPC تلقائياً.

ماذا يحدث عند تفعيل أو تعطيل EKS Auto Mode؟

الحالة	السلوك والتأثير التشغيلي	حالة الاستخدام الموصى بها
تفعيل (Enable)	تتولى AWS إدارة الـ Nodes تلقائياً. لا تحتاج لـ Managed Node Groups أو Fargate. يتم إضافة عقد حساسة للموارد واختيار فئات EC2 المثالية تلقائياً.	المشاريع الجديدة، الفرق التي ترغب في تقليل العبء التشغيلي (Operational Overhead)، والتركيز على التطبيقات فقط.
تعطيل (Disable)	يعود العنقود للنمط التقليدي (Custom/Self-Managed). أنت مسؤول عن اختيار أنواع EC2، إعداد Karpenter/CAS، تحديث الـ AMIs، وضبط الـ Auto Scaling Groups.	المؤسسات التي تتطلب تخصيصاً دقيقاً للغاية لنظام التشغيل، شبكات خاصة جداً، أو استخدام وحدات تخزين وتكوينات غير نمطية.

ملاحظة: يمكنك تفعيل Auto Mode عند إنشاء العنقود أو ترقية العناقيد الحالية، مع الاحتفاظ بإمكانية استخدام Node Groups تقليدية بجانبه إذا لزم الأمر.

CamScanner

2. طرق المصادقة: EKS API vs API and ConfigMap

في السابق، كان EKS يعتمد حصرياً على خريطة إعدادات داخلية تُدعى `aws-auth ConfigMap` لربط هويات AWS IAM بأدوار صلاحيات Kubernetes (RBAC). قدمت AWS ميزة **EKS Access Entries** للتحكم بالمصادقة عبر EKS API مباشرة.

النمط (Access Mode)	طريقة العمل (Architecture)	المميزات والسلبيات
EKS API الخيار الحديث والموصى به	تمتع بإدارة الصلاحيات مباشرة عبر AWS IAM API باستخدام `Access Entries` دون التعديل داخل Kubernetes.	**المميزات:** إدارة آمنة عبر Terraform/CloudFormation، تدقيق ممتازة عبر CloudTrail، لا مجال لخطأ تنسيق YAML. **السلبيات:** يتطلب تحديث الأدوات القديمة.
API and ConfigMap الانتقالي (Hybrid)	يدعم الطريقتين معاً؛ يفحص EKS الـ Access Entries أولاً، وإذا لم يجد الهوية ينتقل للفحص داخل `aws-auth ConfigMap`.	**المميزات:** يسهل عملية الهجرة (Migration) تدريجياً دون كسر الصلاحيات الحالية. **السلبيات:** تعقيد إضافي في تتبع مصدر الصلاحية.
ConfigMap Only القديم (Legacy)	تعتمد فقط على تعديل ملف `aws-auth` في namespace المسمى `kube-system`.	**السلبيات:** عرضة لأخطاء التنسيق، قد تؤدي لقطع الاتصال بالعنقود، صعوبة الأتمتة التجميعية.

كيفية اختيار النمط المناسب والفرق الجوهري

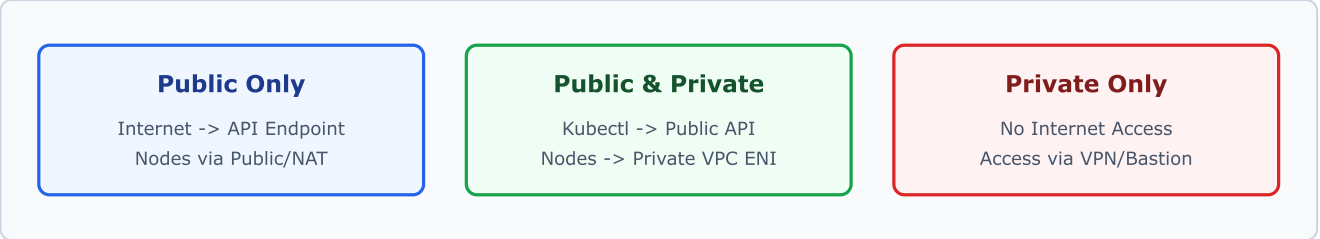
الفرق الأساسي يكمن في **موقع مصدر الحقيقة (Single Source of Truth)**: في نمط **EKS API**، تصبح AWS IAM هي المتحكم الكامل عبر APIs مثل `CreateAccessEntry`، مما يلغي الحاجة للوصول إلى داخل العنقود لتعديل صلاحيات المستخدمين والـ Node Roles. بينما في **ConfigMap**، أي خطأ في كتابة ملف الـ YAML قد يتسبب في قفل العنقود (Lockout) ومنع الأدمن من الدخول.

> **توصية AWS المعتمدة:**
> توصي AWS بالانتقال الكامل إلى نمط **EKS API** وتجريد ملف `aws-auth ConfigMap` نهائياً لرفع مستوى الأمان وسهولة الإدارة عبر أدوات Infrastructure as Code (IaC).

CamScanner

3. طرق الوصول بنقاط النهاية (Cluster Endpoint Access)

يحتوي كل عنقود EKS على نقطة نهاية (API Server Endpoint) يُستخدم بواسطة `kubectl` والـ Worker Nodes للتواصل مع Control Plane. توفر AWS 3 أنماط لعزل أو إتاحة هذا الـ Endpoint:



نمط الـ Endpoint	أين يقع الـ API Endpoint؟	كيف تتصل العقد (Nodes) به؟	كيف يتصل المسؤول (Kubectl)؟
Public	متاح على الإنترانت العام (Public Internet).	تتصل عبر الإنترنت (تحتاج NAT Gateway أو Public IPs).	من أي مكان عبر الإنترنت (يمكن تقييد الـ CIDR Block).
Public and Private الأكثر شيوعاً	متاح عموميًا وداخلياً داخل الـ VPC.	تتصل داخلياً عبر VPC Endpoint ENIs (دون الخروج للإنترنت).	عبر الإنترنت مباشرة دون الحاجة لـ VPN.
Private Only الأعلى أماناً	متاح **فقط** داخل الـ VPC الخاص بالعنقود.	تتصل داخلياً بالكامل عبر مسارات شبكة VPC الخاصة.	يتطلب وجودك داخل الـ VPC (عبر AWS VPN, DirectConnect, أو Bastion Host).

كيفية الاتصال بالعنقود (How to Access):

- **في نمط Public:** ينفذ المستخدم أمر `aws eks update-kubeconfig --name cluster_name` ويتصل مباشرة عبر الإنترنت.
- **في نمط Private:** يجب الاتصال أولاً بشبكة المؤسسة عبر **AWS Client VPN** أو الاتصال بسيرفر قفز **Bastion Host** داخل الـ VPC، ثم تشغيل أمر `kubectl` .

CamScanner

4. حساب عناوين الـ IP لـ Worker Nodes في Kubernetes

حساب عناوين الـ IP في EKS يتوقف بشكل أساسي على **نوع الـ CNI Plugin** المستخدم. افتراضياً، يستخدم EKS محرك **AWS VPC CNI**، والذي يمنح كل Pod عنوان IP حقيقي من الـ Subnet الخاص بالـ VPC، بخلاف شبكات Overlay مثل Calico.

المعادلة الأساسية لحساب الـ IPs لـ AWS VPC CNI:

```
Total IPs Required = Number of Nodes + (Number of Nodes × Max Pods per Node) + Overhead IPs
```

حيث يعتمد عدد الـ Pods المتاحة للـ Node على عدد بطاقات الشبكة (ENIs) وعدد الـ Secondary IPs المتاحة لنوع الـ EC2 Instance.

جدول توضيحي لحساب الـ IP حسب نوع الـ EC2 Instance:

نوع الـ Instance	عدد الـ ENIs	IPs لكل ENI	أقصى عدد Pods للـ Node	إجمالي الـ IPs المطلوب لـ 10 Nodes
`t3.medium`	3	6	**17** = 2 + ((1 - 6) × 3) **Pods**	Nodes + (10 × 17) = **180** 10 **IPs**
`m5.xlarge`	4	15	**58** = 2 + ((1 - 15) × 4) **Pods**	Nodes + (10 × 58) = **590** 10 **IPs**
`c5.4xlarge`	8	30	**234** = 2 + ((1 - 30) × 8) **Pods**	Nodes + (10 × 234) = **2,350** 10 **IPs**

تحذير هام حول استنفاذ عناوين الـ IP (IP Exhaustion):
بما أن VPC CNI يقوم بحجز عناوين IP مسبقاً (Warm ENI Target)، فإن الـ Subnet قد ينفد سريعا. **الحل:** استخدام ميزة *Custom Networking* لتخصيص Secondary CIDR (مثل `100.64.0.0/16`) حصرياً للـ Pods، أو تفعيل ميزة *Prefix Delegation* التي تزيد عدد الـ IPs المتاحة للـ Node بـ 16 ضعفاً!

حسابات النمط الثاني: Overlay Networks (مثل Calico CNI)

إذا استخدمت Calico في نمط Overlay، فإن كل Node تستهلك **IP واحد فقط** من الـ VPC Subnet، وتأخذ الـ Pods عناوين افتراضية داخلية لا تستهلك من شبكة AWS VPC.

CamScanner